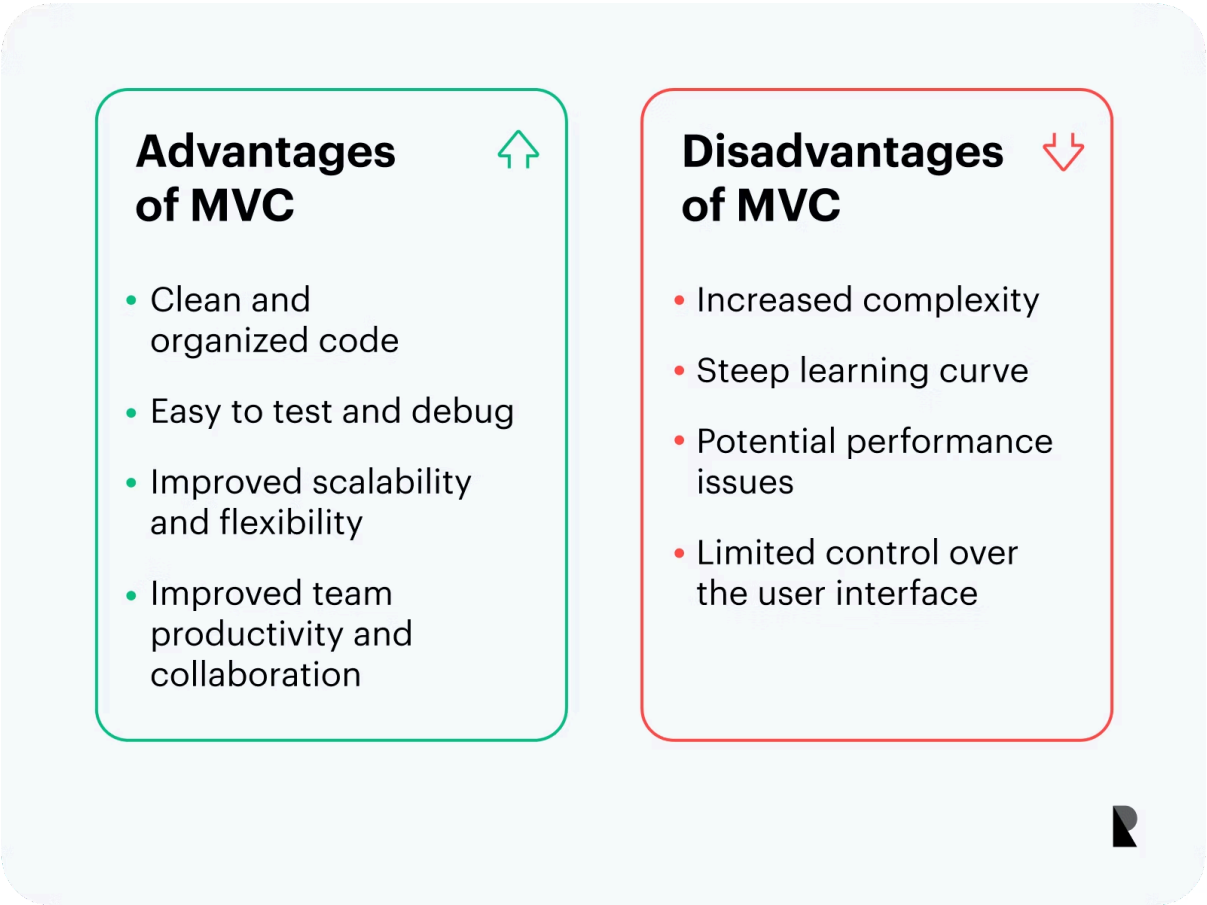
Characteristics of MVC

- It provides a clear distinction between business logic, application logic, and input logic.
- It gives you full control over HTML and URLs, making it easy to design your web application architecture.
- It can be used to build applications with easily understandable and searchable URLs.
- Supports Test-Driven Development.

MVC is a standard design pattern familiar to many programmers due to its scalability and extensibility. MVC is commonly used as a standard web development framework as well as for mobile applications.



Model

The Model is the application component that corresponds to all the logic related to the data domain; in short, it's the back-end containing all the application's data logic. The

data here can be data being passed between the View and Controller components, or any other data related to the business logic.

If the state of this data changes, the Model will usually notify the View (so the screen can change as needed) and sometimes the Controller (if other logic is needed to update the View).

For example, let's say you're developing a shopping app. Here, the Model will specify what data the shopping cart will include—such as items, prices, etc.—and what data is already present in the cart.

Typically, Model objects can be retrieved from a database, manipulated, and their state stored in the database.

View

Views are the components that display the user interface (UI) of an application. Typically, this UI is created from Model data.

For example, in a shopping app, the View defines how the shopping cart is displayed to the user and receives data from the Model to display it. The View includes all UI elements such as buttons, dropdown lists, etc., that the end user interacts with.

Controller

Controllers are components that handle user interaction to work with the Model (updating data logic) and/or with the View (updating the user interface display).

In an MVC application, the Controller processes query string values and passes these values to the Model, which then queries the database using those values. The View displays the information processed by the Controller and responds to input from user interaction.

For example, in a shopping app, you could add buttons to the user's shopping cart that allow them to add or remove items.

These user actions require the Model to be updated; therefore, input is sent to the Controller, which then manipulates the Model accordingly, and finally sends the updated data to the View.

MVC helps you create separate applications for different aspects of the application (input logic, business logic, and user interface logic), while providing connectivity between these components.

The MVC model specifies the location of each type of logic in the application:

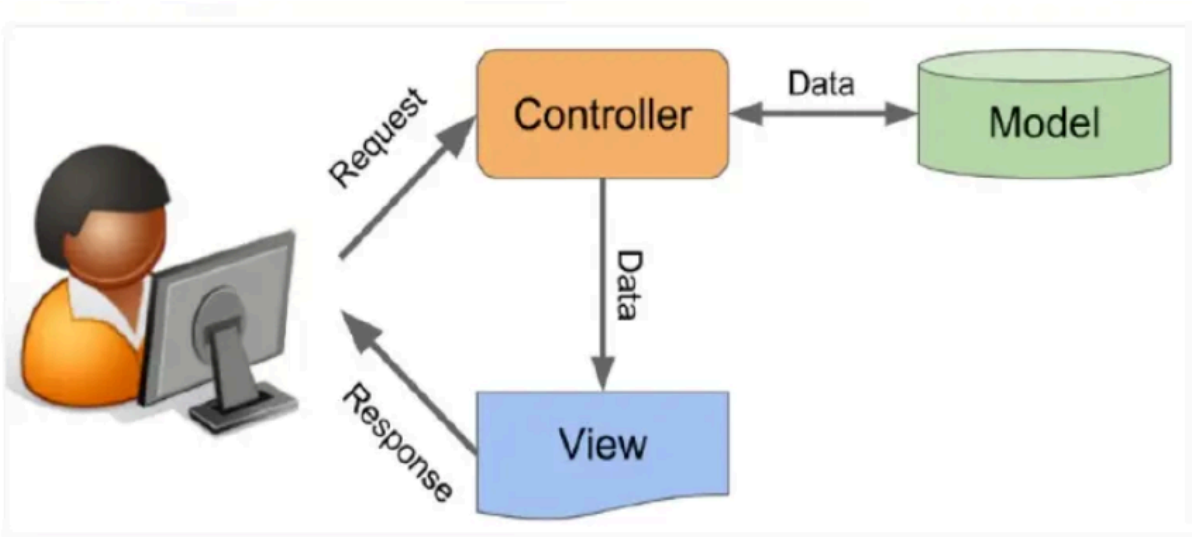
- The core business logic is the Model.
- User interface logic belongs to the View.
- The input logic belongs to the Controller.

This separation helps you manage the complexity when building an application because it allows you to focus on one aspect of the implementation at a time.

For example, you can focus on the user interface display without relying on business logic.

The combination of the three main components of an MVC application also promotes parallel development. For example, one programmer can work on the View, a second programmer can work on the Controller logic, and a third programmer can focus on the business logic in the Model.

Conceptually, each approach to MVC development is similar in that they all strive to adhere to the principle of Separation of Concerns (SoC), a design model that divides the application into separate units with minimal functional overlap.



Why should we use the MVC model?

Easily organize large-scale web applications.

Because of the code separation across the three levels, it becomes extremely easy to divide and organize web application logic into large-scale applications (which need to be managed by large teams of programmers).

The main advantage of using such coding practices is that it helps to quickly identify specific code sections and allows for easy addition of new functionality.

Supports asynchronous method invocation.

Because the MVC architecture works well withJavaScriptAnd since it's a JavaScript framework, it's no surprise that it also supports the use of Asynchronous Method Calls (AMI), allowing developers to build web applications that load faster.

This means that MVC applications can be created to work even with PDF files, web-specific browsers, and desktop utilities.

Easy to modify

Using the MVC approach allows for easy modification of the entire application. Adding/updating new view types is simplified in the MVC pattern (because each part is independent of the others).

Therefore, any changes to a specific part of the application will never affect the entire architecture. Conversely, this will help increase the flexibility and scalability of the application.

The programming process is faster.

Because of the code separation between the three levels, developing web applications using the MVC model allows one programmer to work on a specific part (say, the view) while another programmer can work simultaneously on any other part (say, the controller).

This allows for easy implementation of business logic and helps speed up the programming process by four times. It has been found that, compared to other development models, the MVC model shows higher development speeds (up to three times faster).

Easy planning and maintenance

MVC is very useful in the initial planning phase of an application because it provides programmers with an outline of how to organize their ideas into actual code.

MVC is also a great tool for limiting code duplication and allowing for easy application maintenance.

Returns unformatted code data.

By returning unformatted data, MVC allows you to create your own view engine. For example, any data type can be formatted using HTML, but with MVC, you can also format the data using Macromedia Flash or DreamViewer.

This advantage is very useful for programmers because similar components can be reused with any interface.

Supports TTD (Test-Based Programming)

The main advantage of the MVC pattern is that it greatly simplifies the testing process. It makes debugging large-scale applications easier because many levels are

structurally defined and precisely written within the application. Therefore, programming an application using unit tests is trouble-free.

SEO-friendly platform

The MVC framework greatly supports the development of SEO-friendly web applications. To generate more traffic from a specific application, MVC provides an easy way to develop SEO-friendly RESTful URLs.

Disadvantages of MVC

Complexity

MVC can increase codebase complexity because it requires programmers to separate their code into three distinct components: Model, View, and Controller.

This separation can lead to more files, classes, and instructions in the code, which can make the application harder for programmers to understand, especially for smaller or simpler projects. The need to manage interactions between these components can also add to the complexity.

Learning curve

Understanding and implementing MVC can be challenging, especially for programmers new to the MVC model. This model requires a shift in software design and structuring. Programmers must learn to think in terms of Model, View, and Controller, and understand how these components interact with each other.

This learning curve can slow down development and lead to mistakes in the early stages of MVC implementation.

Maintain the difficulty.

Over time, maintaining an MVC application can become more challenging. Without proper documentation and clear coding standards, it can be difficult to track how the different components interact. As the project grows, managing changes and updates to the MVC can become more cumbersome. This can lead to increased development time and costs.

Inflexibility

MVC can be a bit rigid in certain situations. It's best suited for applications with clearly defined requirements and a clear understanding of how data flows through the system.

In dynamic or rapidly changing projects where development requirements evolve quickly, MVC may not be the most flexible or adaptable model. Implementing significant changes to the architecture can be challenging.

The number of files has increased.

Implementing the MVC model often results in an increased number of files in a project. Each component is typically represented in separate files or classes.

This can be beneficial for large, complex applications, but for smaller projects, it can lead to an unnecessarily large number of files, making the project more difficult to organize and manage.